

# Kamień milowy 5 - aplikacja demonstracyjna i próba wdrożenia modelu

## Dane autora

Imię i nazwisko: Mateusz Adamczak

Numer albumu / grupa: 423062, projekt realizowany samodzielnie

Temat projektu: NaturaVision - lokalny model multimodalny do rozpoznawania roślin i grzybów leśnych

Data przygotowania sprawozdania: 21.05.2026

Zakres dokumentu: ten kamień milowy opisuje próbę uruchomienia modelu w aplikacji mobilnej, powód rezygnacji z tej ścieżki w obecnym etapie oraz przygotowanie stabilniejszej aplikacji desktopowej. Nie powtarzam tutaj szczegółów treningu, datasetu ani wykresów z poprzednich kamieni milowych.

## 1. Cel kamienia milowego

Celem tego etapu było przejście od wytrenowanego checkpointu do aplikacji, która można pokazać jako działający prototyp. Początkowo zakładałem aplikację mobilną, ponieważ taki sposób pracy najlepiej pasuje do rozpoznawania organizmów w terenie. W praktyce największym ryzykiem okazał się lokalny runtime modelu multimodalnego na telefonie.

Po diagnostyce zdecydowałem się przygotować aplikację desktopową. Ta decyzja nie oznacza porzucenia aplikacji mobilnej jako pomysłu docelowego, ale była bardziej realistyczna dla obecnego kamienia milowego: desktop pozwala pokazać model w działaniu na sprawdzonym sprzęcie, bez ukrywania problemów z backendem Android/Vulkan.

## 2. Prototyp aplikacji mobilnej

Przygotowałem aplikację Android w Kotlinie i Jetpack Compose. Aplikacja miała:

- wybór obrazu z galerii,
- obsługę szybkiego zdjęcia z aparatu,
- ekran wyniku z `label_id`, nazwa polska, nazwa łacińska, nazwa angielska i królestwem,
- lokalny katalog wszystkich klas modelu,
- parser odpowiedzi w formacie `{"label_id": "..."}` ,
- dedykowany przycisk test suite do sprawdzania gotowości aplikacji i runnera.

Dodatkowo przygotowałem strukturę pod realny lokalny model GGUF. Aplikacja szukała paczki modelu, projektora obrazu i lokalnego runnera. Po stronie natywnej próbowałem użyć `llama.cpp` / `mtmd` przez JNI, czyli bez wysyłania obrazu do zewnętrznej usługi.

#### Screenshot 1 - Android test suite

Brak pliku: Screenshots/km5/01\_android\_test\_suite.png

**Zrzut 1.** Test suite w aplikacji Android. Screenshot powinien pokazać ekran testów, status runnera oraz ostrzeżenia związane z lokalnym backendem.

### 3. Co nie zadziało na telefonie

Testy na telefonie zacząłem od ścieżki CPU, ponieważ była najprostsza i najbardziej przewidywalna diagnostycznie. Ten wariant miał sens jako punkt odniesienia, ale dla lokalnego modelu multimodalnego klasy `4B` był zbyt wolny do sensownej demonstracji. Z tego powodu przeszedłem na akcelerację przez GPU Adreno z backendem Vulkan. Ten krok był logiczny, bo pozwalał zachować najważniejsze założenie projektu, czyli inferencje lokalna na urządzeniu, a jednocześnie próbował wykorzystać sprzętową akcelerację telefonu zamiast liczyć tylko na procesor.

Problemy pojawiły się dopiero po przejściu na ścieżkę Adreno/Vulkan. Aplikacja potrafiła uruchomić warstwę testową, a model oraz projektor były odnajdywane. Backend zaczynał generację, ale odpowiedź modelu ulegała degeneracji. Dlatego najważniejszy problem nie wyglądał na zwykły brak pamięci RAM, tylko na niestabilność konkretnej ścieżki uruchomieniowej.

Najważniejsze obserwacje po przejściu na Vulkan/Adreno:

- profil `phone_vulkan_current` generował odpowiedź w stylu `{"label_id":"!!!!!!!!!!..."}`,
- profil `phone_vulkan_no_flash` zmieniał błąd na powtarzające się spacje, ale nadal nie dawał poprawnej klasy,
- zmiana flash attention zmieniała typ degeneracji, lecz nie rozwiązywała problemu,
- przy zwiększaniu liczby tokenów obrazu rosło ryzyko niestabilności backendu,
- wymagania modelu multimodalnego były trudniejsze niż dla zwykłego modelu tekstowego.

Wniosek: problem był związany głównie ze ścieżką backendu `Android + Vulkan/Adreno + llama.cpp mtmd`. CPU było zbyt wolne jako docelowa demonstracja, a Adreno/Vulkan dawało potrzebną akcelerację, ale w praktyce okazało się niestabilne dla tego konkretnego multimodalnego modelu i konfiguracji. Była to zbyt niszowa ścieżka uruchomieniowa: lokalny model Qwen3.5 multimodalny, własny eksport GGUF, `llama.cpp mtmd`, Android i Vulkan/Adreno nie miały wystarczająco dojrzałego wsparcia, żeby traktować telefon jako pewną platformę demonstracyjną.

#### Screenshot 2 - Android runner warning

Brak pliku: Screenshots/km5/02\_android\_runner\_warning.png

**Zrzut 2.** Ostrzeżenie lub log diagnostyczny runnera na telefonie. Screenshot powinien pokazać, że aplikacja dochodzi do etapu lokalnego backendu, ale wynik nie jest jeszcze stabilny.

---

## 4. Dlaczego wybrałem desktop zamiast finalizowania mobile

Zdecydowałem, że w tym kamieniu milowym nie będę finalizował aplikacji mobilnej, ponieważ głównym celem tematu nie była sama aplikacja na telefon, tylko lokalny model rozpoznający rośliny i grzyby. Aplikacja mobilna była jednym z możliwych sposobów pokazania modelu, ale nie powinna zastąpić głównego celu projektu. Jeśli telefonowy runtime nie działał stabilnie, to dalsze rozwijanie klienta Android nie rozwiązywało najważniejszego problemu.

W tej sytuacji lepszym wyborem było przygotowanie aplikacji desktopowej, która nadal spełnia główne założenie: model działa lokalnie, bez wysyłania obrazu do zewnętrznej usługi. Desktop pozwolił pokazać realną inferencję na sprawdzonym sprzęcie, zamiast budować demonstrację wokół niestabilnej ścieżki mobilnej.

Najważniejsze powody decyzji:

- telefonowy backend Vulkan zwracał zdegenerowane tokeny zamiast poprawnego JSON-a,
- Qwen3.5 jako model multimodalny jest bardziej wrażliwy na backend niż prostsze modele tekstowe,
- stabilna inferencja wymagała wysokiej liczby tokenów obrazu, co pogarszało sytuację na telefonie,
- utrzymywanie aplikacji mobilnej jako głównego demo wymagałoby osobnego portu runtime, np. CPU/OpenCL albo mniejszego modelu,
- desktop pozwalał pokazać ten sam model w bardziej kontrolowanych warunkach,
- lokalna inferencja pozostała zachowana, czyli najważniejsze założenie projektu nie zostało zmienione.

To była decyzja inżynierska: nie zmieniałem celu projektu, tylko zmieniłem platformę demonstracyjną. Zamiast maskować problem w aplikacji telefonicznej, pokazałem działający lokalny model w wariantcie desktopowym i opisałem, dlaczego mobilny runtime wymaga osobnego etapu prac.

---

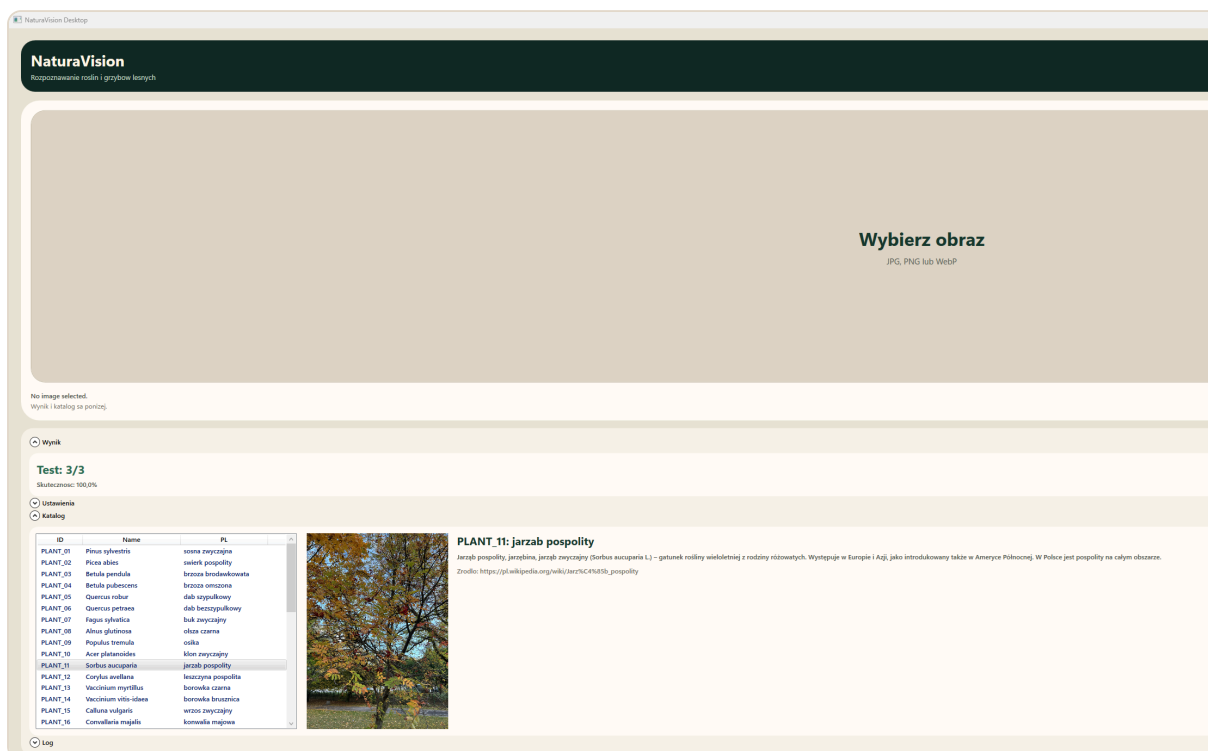
## 5. Aplikacja desktopowa

Po decyzji o zmianie ścieżki przygotowałem aplikację desktopową w WPF. Jej głównym założeniem jest prosty przepływ: użytkownik wybiera obraz albo robi zrzut fragmentu ekranu, uruchamia rozpoznawanie i dostaje wynik z opisem gatunku.

Najważniejsze funkcje aplikacji desktopowej:

- duże pole obrazu jako centralny element ekranu,
- przycisk **Wybierz** do wczytywania pliku,
- przycisk **Zrzut ekranu**, który korzysta z natywnego narzędzia Windows do wycinania fragmentu ekranu,
- przycisk **Rozpoznaj** do uruchomienia modelu,

- zakładka **Wynik** z przewidzianym **label\_id**, nazwa polska, nazwa łacińska, zdjęciem i krótkim opisem,
- zakładka **Katalog** z pełną taksonomią,
- ukryte domyślnie ustawienia runtime,
- przycisk test suite do szybkiego sprawdzenia kilku przykładów z **test.jsonl**.



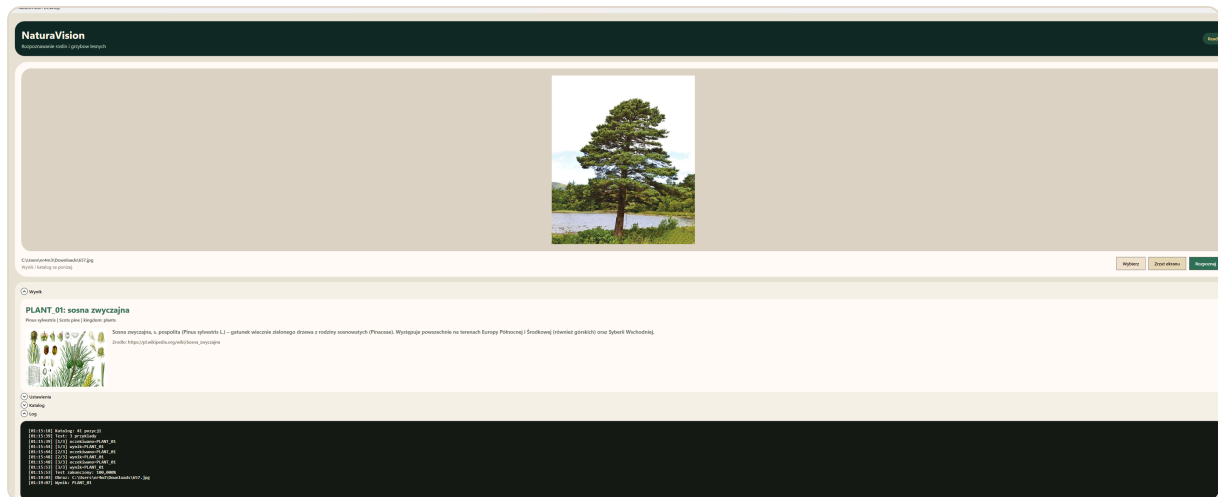
**Zrzut 3.** Ekran startowy aplikacji desktopowej. Screenshot powinien pokazać duże pole wyboru obrazu i podstawowe przyciski.

## 6. Integracja modelu na desktopie

Aplikacja desktopowa uruchamia model przez **llama-mtmd-cli.exe**. Zweryfikowana konfiguracja na moim komputerze używa:

- Windows x64,
- RTX 4070,
- CUDA build **llama.cpp**,
- modelu **forest-taxa-qwen35-4b-q4\_k\_m-fixed.gguf**,
- projektora **forest-taxa-qwen35-4b-mmproj-f16.gguf**,
- **GPU layers = 99**,
- **image tokens = 1024**,
- deterministycznego dekodowania,
- schematu JSON ograniczającego wyjście do dozwolonych **label\_id**.

Ważna decyzja techniczna: w repozytorium nie przechowuje wag modelu. Są one za duże i powinny być pobierane osobno z Hugging Face. Repo zawiera kod, dokumentację, katalog gatunków i instrukcje uruchomienia, ale nie pełne checkpointy.



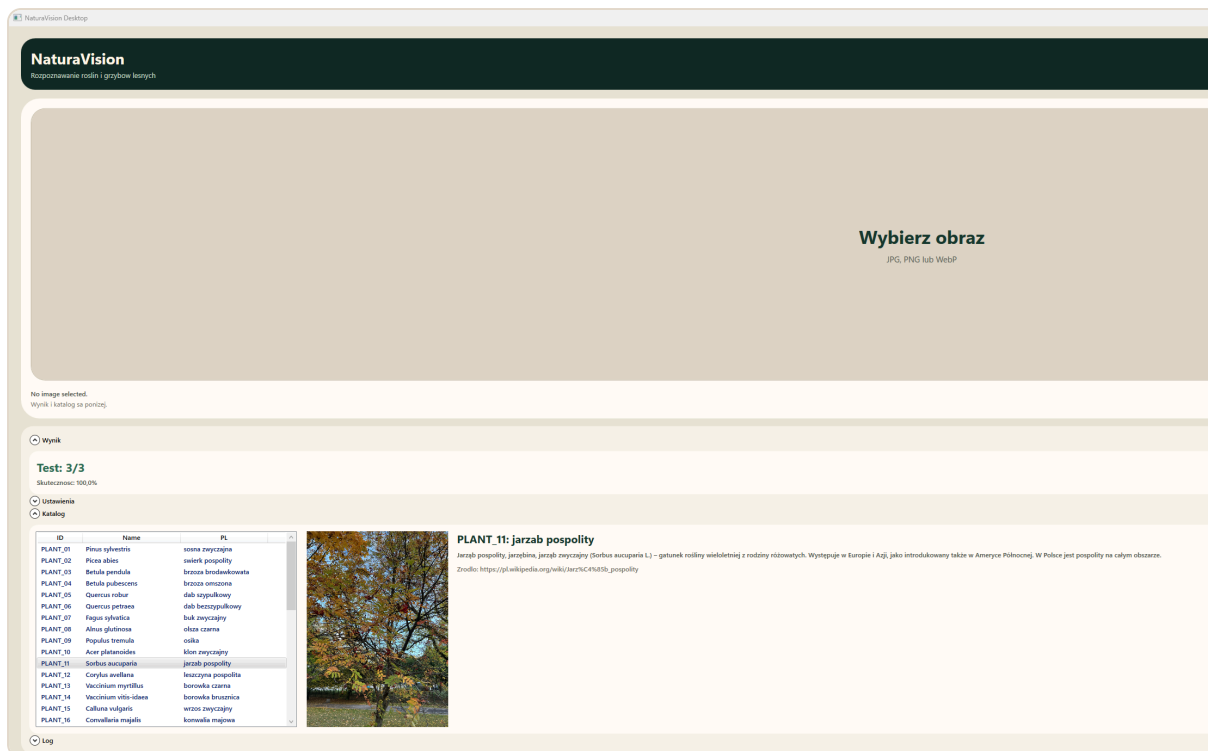
**Zrzut 4.** Wynik rozpoznawania w aplikacji desktopowej. Screenshot powinien pokazać przewidziana klasę, zdjęcie gatunku, opis i źródło.

## 7. Katalog gatunkow i opisy

Do aplikacji dodałem lokalny katalog mediów dla wszystkich klas. Dla każdej rośliny i każdego grzyba aplikacja ma:

- `label_id`,
- nazwe polska,
- nazwe lacinska,
- nazwe angielska,
- krotki opis po polsku,
- link do zrodla,
- jedno zdjecie zapisane lokalnie.

Opisy pochodzą z polskiej Wikipedii. Zdjęcia są cache'owane lokalnie: tam, gdzie Wikimedia pozwoliła pobrać miniatury, zapisane są obrazy z Wikipedii/Wikimedia; tam, gdzie serwer ograniczył pobieranie przez **429 Too many requests**, został użyty lokalny fallback z datasetu. Dzięki temu aplikacja nie zależy od internetu podczas demonstracji.

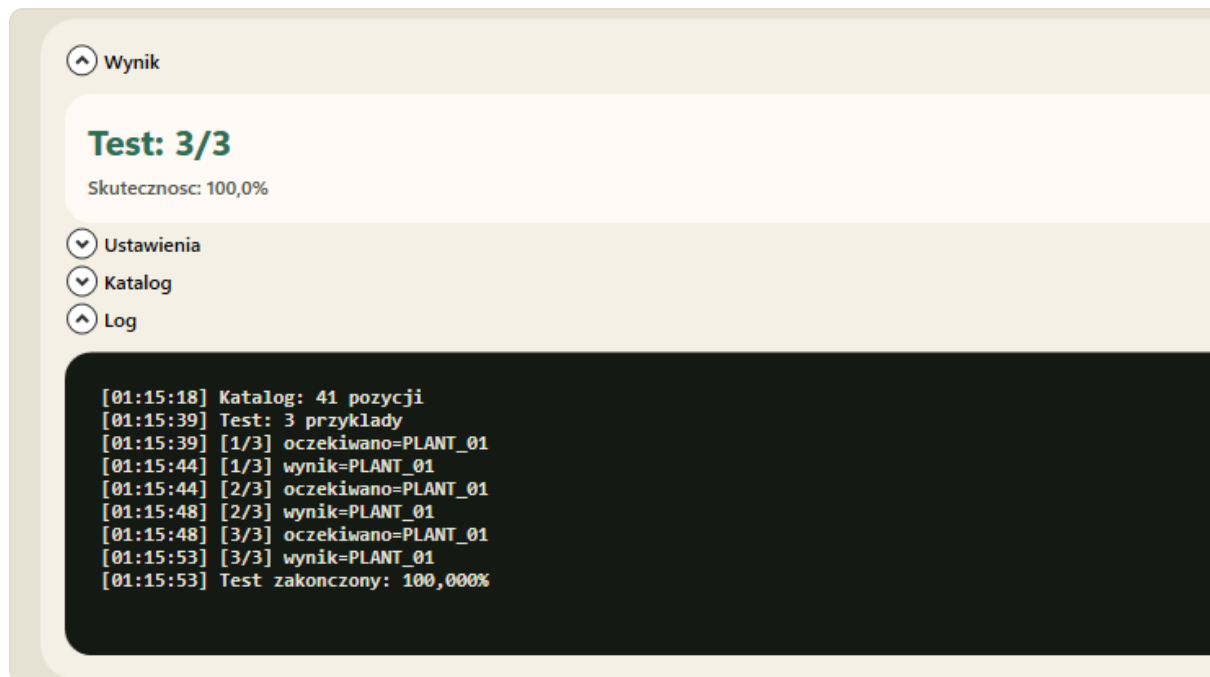


**Zrzut 5.** Zakładka katalogu gatunków. Screenshot powinien pokazać listę klas oraz opis i obraz wybranego gatunku.

## 8. Walidacja wykonanej pracy

Wykonane sprawdzenia:

- aplikacja desktopowa kompiluje się przez `dotnet build`,
- katalog `data/taxonomy_media` zawiera opisy i obrazy dla 40 publicznych klas,
- aplikacja ma ukryte domyślnie ustawienia runtime, żeby główny ekran był prosty,
- wynik predykcji jest mapowany z `label_id` na pełne informacje o gatunku,
- README opisuje, jak pobrać wagi i uruchomić aplikację.



**Zrzut 6.** Test suite w aplikacji desktopowej. Screenshot pokazuje krotki test na trzech przykładach, w którym aplikacja poprawnie odczytała oczekiwane klasy i uzyskała wynik **3/3**.

Możliwe dalsze rozszerzenie dokumentu:

- opcjonalnie wygenerować PDF po dodaniu obrazów.

## 9. Wnioski

Ten kamień milowy był mniej o samym trenowaniu, a bardziej o praktycznym wdrożeniu modelu. Najważniejsza lekcja jest taka, że dobry wynik checkpointu nie oznacza automatycznie, że model będzie łatwy do uruchomienia na każdym urządzeniu. W przypadku aplikacji mobilnej ograniczeniem okazał się bardzo niszowy runtime multimodalny bez wystarczająco stabilnego wsparcia Vulkan/Adreno dla tej konfiguracji.

Ostatecznie przygotowałem stabilniejszą aplikację desktopową, która lepiej nadaje się do demonstracji obecnego stanu projektu. Aplikacja pokazuje cały przepływ: obraz wejściowy, lokalna inferencja, wynik klasyfikacji, opis gatunku i katalog taksonomii. Mobilna ścieżka zostaje jako możliwy kierunek dalszego rozwoju, ale wymaga osobnego portu i testów backendu.